SYMBIOSIS INSTITUTE OF TECHNOLOGY, HYDERABAD
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of India)
Re-Accredited by NAAC with 'A++' Grade

Skin Segmentation using Machine Learning: A Comparative Study of Pixel-Level Skin and Non-Skin Classification

CASE STUDY (CA-4)

Subject: Supervised Learning

Submitted By:
P.V. Lakshmi Srujana - 24070721025

Submitted To:
Dr. Bibhuprasad Sahu
Assistant Professor
Department of Computer Science and Engineering

Date of Submission: 12 November 2026

Abstract

Detecting skin pixels is a fundamental pre-processing step in many computer vision applications, including face and hand localisation, gesture-based interfaces, and image filtering. This paper presents a comparative machine learning study of pixel-level skin and non-skin classification using the UCI Skin Segmentation dataset. Each observation is described by three colour-intensity features, Blue (B), Green (G) and Red (R), and a binary label indicating whether the pixel is skin or non-skin. The dataset contains 245,057 observations, of which 50,859 are skin pixels and 194,198 are non-skin pixels. Exploratory analysis examined class balance, channel distributions, inter-channel correlation, duplicate rows and label consistency, and revealed 193,613 duplicate rows and 11 colour triplets that carry conflicting labels. To prevent identical colour triplets from appearing in both training and test data, a group-aware, stratified split was used and verified to have zero overlapping groups.

Eight classifiers were evaluated: Logistic Regression, Decision Tree, K-Nearest Neighbors (KNN), Random Forest, Histogram-based Gradient Boosting, XGBoost, Linear Support Vector Classification (LinearSVC) and a Multilayer Perceptron (MLP). Performance was measured using accuracy, precision, recall, F1-score, confusion matrices and ROC-AUC. The study further applied five-fold StratifiedGroupKFold cross-validation, randomised hyperparameter search, and three class-imbalance strategies (class weighting, random oversampling and random undersampling). In the final cross-validated comparison, Random Forest obtained the highest mean F1-score of 0.998865, narrowly ahead of KNN at 0.998851, and was selected as the final model. On the held-out test set it achieved 99.92% accuracy, 99.70% precision, 99.94% recall, an F1-score of 0.9982 and a ROC-AUC of 0.99998. Feature-importance analysis using built-in tree importance and permutation importance consistently identified the red channel as the most influential feature. These scores describe performance on this dataset and evaluation design only; they do not by themselves establish robustness to lighting changes or fairness across skin tones.

Keywords: *skin segmentation, pixel classification, RGB features, Random Forest, class imbalance, cross-validation, feature importance.*

Introduction

Skin segmentation is a pixel-level classification problem in computer vision: given an image, the goal is to decide which pixels belong to human skin and which do not. A reliable skin mask is a useful first stage in many pipelines, such as locating faces and hands, supporting gesture-based human-computer interaction, and restricting later processing to regions of interest. Because the decision is made for every pixel, the method must be accurate, and, for interactive use, fast.

This project treats skin segmentation as a supervised binary classification task. Rather than training an image-to-mask network, it uses a tabular representation in which every pixel is a three-dimensional BGR feature vector and the classifier learns a mapping from that vector to skin or non-skin. This makes it possible to compare a broad range of classical algorithms, from

a linear baseline to tree ensembles, boosting and a small neural network, under one consistent experimental protocol.

Two aspects make this dataset more subtle than its very high headline scores suggest. First, the data is imbalanced, with roughly four non-skin pixels for every skin pixel, so accuracy alone can be misleading. Second, it contains a very large number of repeated colour triplets, so a naive random split would allow the same triplet to appear in both training and test data. The project therefore uses a group-aware split and group-aware cross-validation, and it reports its limitations explicitly. The experiments evaluate pixel classification from RGB values only; they do not exploit neighbouring pixels, reconstruct full images, or test performance across annotated skin-tone groups.

Case Studies: The Stakes of the Problem

To appreciate why accurate and carefully evaluated skin detection matters, we consider two scenarios in which skin-related visual analysis plays a central role.

Case Study 1: Skin Detection in Hand Tracking and Gesture Interfaces

Vision-based interfaces, such as gesture control of devices, sign-language assistance, and touch-free interaction, must first find the user's hands in a video frame. Skin colour is a fast and cheap cue for proposing candidate hand regions before heavier shape or motion analysis is applied. In such a pipeline, the two error types have different consequences. False negatives (skin pixels labelled as background) fragment the hand mask and can cause tracking to be lost, while false positives (background pixels labelled as skin) add clutter such as wooden furniture or clothing with skin-like tones and mislead later stages. This scenario motivates reporting precision and recall separately, rather than relying on overall accuracy, and it motivates favouring models with fast inference.

Case Study 2: Skin-Tone Bias in Computer Vision Systems

Buolamwini and Gebru (2018) audited commercial gender-classification systems and reported error rates of up to 34.7% for darker-skinned women, compared with a maximum error rate of 0.8% for lighter-skinned men. Although that study concerned gender classification rather than skin segmentation, it demonstrated how a vision system that appears accurate overall can perform very unevenly across skin tones when its training and evaluation data are not representative. For skin segmentation, this is a direct warning: a classifier trained on colour values alone may be tuned to the skin tones that dominate its data. This case study motivates the explicit caution in this report that the supplied dataset has no skin-tone annotations, and that high aggregate scores therefore cannot be read as evidence of fairness.

Problem Statement

Let a pixel be represented by the feature vector $x = [B, G, R]$, where each component is an integer colour intensity between 0 and 255, and let y indicate whether the pixel is skin ($y = 1$) or non-skin ($y = 0$). The task is to learn a function $f(x)$ that predicts y from the three colour values. In the dataset, the original labels are 1 for skin and 2 for non-skin; these are mapped so

that skin is the positive class, which makes precision, recall and F1-score interpretable with respect to skin pixels.

Several practical difficulties complicate this task. The two classes overlap heavily in the blue and green channels, so no single channel separates them cleanly. The classes are imbalanced, which can mislead accuracy-based evaluation. The data contains many duplicate colour triplets, and a small number of triplets appear with both labels, so the mapping from colour to class is not perfectly unique. Finally, evaluation that ignores these duplicates can overstate performance because identical feature vectors may appear in both the training and test sets. There is therefore a need for a carefully validated, comparative study that identifies a reliable, efficient and interpretable classifier while being honest about what the measured scores do and do not show.

Objectives

Drawing from the problem statement and the case studies above, this project establishes the following core objectives:

- To inspect the dataset and understand its class distribution, feature ranges, inter-feature relationships, duplicate rows and label consistency.
- To build a stratified, group-aware train-test partition so that identical BGR combinations never appear in both training and test data.
- To establish baseline performance using Logistic Regression, Decision Tree and KNN, and to extend the comparison to Random Forest, Histogram-based Gradient Boosting, XGBoost, LinearSVC and MLP.
- To evaluate every model using accuracy, precision, recall, F1-score, confusion matrices and ROC-AUC.
- To apply five-fold group-aware cross-validation and randomised hyperparameter search to assess stability and identify competitive configurations.
- To compare class weighting, random oversampling and random undersampling on selected algorithms.
- To interpret the contribution of each colour channel using built-in tree importance and permutation importance.
- To select a final model based on a balanced assessment of predictive performance, stability, efficiency and interpretability, and to document the limitations of the study.

Literature Review

Skin detection from colour has a long history in computer vision. Early work modelled skin colour directly in colour space. Jones and Rehg (2002) built statistical colour models from a very large collection of labelled web images and showed that histogram and mixture models of skin and non-skin colour could separate the two classes with useful accuracy. Surveys by Vezhnevets et al. (2003) and Kakumanu et al. (2007) catalogued pixel-based skin detection techniques, including explicit threshold rules in colour spaces such as HSV and YCbCr, non-

parametric histogram methods and parametric Gaussian models. These surveys also highlighted persistent weaknesses, notably sensitivity to illumination and camera characteristics, and the overlap between skin and skin-coloured background objects.

With the growth of machine learning, skin segmentation has been treated as a standard supervised classification problem. The UCI Skin Segmentation dataset, contributed by Bhatt and Dhall, provides labelled BGR samples and is widely used to benchmark classifiers because it is large, has only three features, and is easy to reproduce. On such low-dimensional, densely sampled data, instance-based and tree-based learners are natural candidates: KNN (Cover and Hart, 1967) classifies by local similarity in colour space, Random Forests (Breiman, 2001) average many decorrelated decision trees, and gradient-boosted trees, including the scalable XGBoost system (Chen and Guestrin, 2016), build additive ensembles of shallow trees. Linear models such as Logistic Regression and linear support vector machines provide useful baselines, but a linear boundary in RGB space is not expected to capture the curved region that skin colours occupy.

A recurring methodological concern is evaluation. Datasets in which many samples share identical feature values can inflate test scores if those samples are split at random between training and test sets. Group-aware cross-validation addresses this by keeping all samples of a group together. Class imbalance is a second concern: techniques such as cost-sensitive learning, random oversampling and random undersampling are commonly compared, and their benefit depends on the learner. Finally, there is growing recognition that aggregate accuracy can hide uneven performance across demographic groups (Buolamwini and Gebru, 2018). The present study combines these threads: it compares eight classifiers under a group-aware protocol, examines imbalance handling and feature importance, and states clearly which questions, such as fairness and lighting robustness, remain outside the scope of the available data.

Methodology

The study follows a sequential supervised-learning methodology consisting of data acquisition, exploratory analysis, preprocessing and splitting, baseline modelling, advanced modelling, cross-validation and tuning, class-imbalance experiments, feature-importance analysis, and final model selection.

The first stage loads the UCI Skin Segmentation file and inspects its shape, types, missing values, duplicates and class balance. Exploratory plots compare the distribution of each colour channel between the skin and non-skin classes and examine correlation among the channels. The second stage separates the features (B, G, R) from the target, maps skin to the positive class, and builds a stratified, group-aware train-test split in which every identical BGR triplet forms one group. Features are then standardised using statistics from the training data only.

The third stage trains three baseline models on the same split, followed by five advanced models. All models are evaluated on the held-out test set using accuracy, precision, recall, F1-score, confusion matrices and, where scores are available, ROC-AUC. The fourth stage applies five-fold StratifiedGroupKFold cross-validation with randomised hyperparameter search to the

advanced models. The fifth stage compares class weighting, random oversampling and random undersampling for Logistic Regression, Random Forest and XGBoost. The sixth stage analyses the trained Random Forest and XGBoost models using built-in and permutation feature importance.

Finally, all eight classifiers are compared under the same five-fold group-aware cross-validation on the training partition, and the model with the best balance of cross-validated F1-score, precision, ROC-AUC, stability and interpretability is selected. The selected model is retrained on the full training partition and evaluated once on the held-out test set.

Datasets

UCI Skin Segmentation Dataset

The dataset used in this project is the UCI Skin Segmentation dataset, supplied in the file Skin_NonSkin.txt. It is a tabular collection of pixel-level colour observations rather than a set of complete images with spatial coordinates. The file contains 245,057 rows and four columns: B, G, R and Target. The first three columns are integer colour intensities between 0 and 255, and Target is the class label, where 1 denotes skin and 2 denotes non-skin. Inspection found no missing values in any column.

The dataset is imbalanced: 50,859 observations (20.75%) are skin and 194,198 observations (79.25%) are non-skin, a ratio of approximately 3.84 to 1. This imbalance is the reason that recall, precision and F1-score are reported alongside accuracy.

Attribute	Details
Source	UCI Machine Learning Repository: Skin Segmentation (Skin_NonSkin.txt)
Total Samples	245,057 pixel observations
Skin Pixels	50,859 (20.75%), original label 1
Non-skin Pixels	194,198 (79.25%), original label 2
Features	B, G, R colour intensities (integers, 0-255)
Target Encoding	Skin = 1 (positive class), Non-skin = 0
Missing Values	None
Duplicate Rows	193,613
Unique BGR Combinations	51,433
Conflicting-label Combinations	11
Imbalance Ratio (train)	3.84 : 1 (non-skin : skin)
Train Split	196,319 samples (80.12%)
Test Split	48,738 samples (19.88%), group-aware stratified
Scaling	StandardScaler, fitted on training data only

Table 1: Summary of the UCI Skin Segmentation Dataset

Descriptive Statistics

Statistic	B	G	R	Target
Count	245,057	245,057	245,057	245,057
Mean	125.065	132.507	123.177	1.792
Std. deviation	62.256	59.941	72.562	0.406
Minimum	0	0	0	1
25th percentile	68	87	70	2
Median	139	153	128	2
75th percentile	176	177	164	2
Maximum	255	255	255	2

Table 2: Descriptive Statistics of the Dataset

Duplicates and Conflicting Labels

The inspection found 193,613 duplicate rows but only 51,433 unique BGR combinations. Repeated triplets are expected in pixel data because many pixels share the same colour. However, 11 BGR combinations are associated with both labels, which means that the three colour values do not determine the class perfectly in every case. These findings directly shaped the splitting strategy: if identical triplets appeared in both partitions, a model could be rewarded for memorising colours seen in training. Grouping identical triplets and keeping each group within a single partition reduces this form of train-test overlap.

Implementation

The project was implemented in Python in a Google Colab notebook using pandas, NumPy, scikit-learn, XGBoost, imbalanced-learn, Matplotlib, Seaborn and joblib. The implementation consists of two complementary modelling pipelines that share the same preprocessing and the same group-aware split. Pipeline 1 establishes baseline performance with three classical classifiers. Pipeline 2 extends the study to ensembles, boosting, a linear SVM and a neural network, followed by cross-validation, tuning, imbalance experiments and interpretability analysis.

Data Loading and Preprocessing

The data was read directly from the UCI repository into a pandas DataFrame with columns B, G, R and Target. The feature matrix X was formed from the three colour columns, and the target vector y was created by mapping original label 1 to 1 (skin) and original label 2 to 0 (non-skin). Groups were then assigned by giving every row with the same BGR triplet the same group identifier.

A StratifiedGroupKFold splitter configured with five folds, shuffling enabled and a random seed of 42 was used, and its first split provided the training and test partitions. Stratification preserves class proportions, while grouping prevents any BGR triplet from appearing in both partitions. The resulting partitions contain 196,319 training and 48,738 test samples, and a direct check confirmed that the number of overlapping groups is zero.

```
from sklearn.model_selection import StratifiedGroupKFold

# Group identical BGR combinations together
groups = df.groupby(["B", "G", "R"], sort=False).ngroup()

# Create a stratified, group-aware split
sgkf = StratifiedGroupKFold(n_splits=5, shuffle=True, random_state=42)
train_idx, test_idx = next(sgkf.split(X, y, groups))

X_train, X_test = X.iloc[train_idx], X.iloc[test_idx]
y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]
```

Figure 1: Group-Aware Stratified Split Implementation

Partition	Total	Skin	Non-skin	Skin share
Training	196,319	40,542	155,777	20.65%
Testing	48,738	10,317	38,421	21.17%

Table 3: Group-Aware Train-Test Partition

StandardScaler was fitted on the training features only and then applied to both partitions, so that no test-set statistics influence preprocessing. Scaling matters for distance-based and gradient-based models such as KNN, Logistic Regression, LinearSVC and MLP, whereas tree-based models do not require it and were trained on the original intensities.

Pipeline 1: Baseline Models

The first pipeline trains three baseline classifiers on the same split: Logistic Regression (maximum 1,000 iterations, linear decision boundary), a Decision Tree (default settings) and KNN (k = 5 using a kd-tree search). Every model was fitted with a fixed random seed of 42 where applicable, and training time and prediction time were recorded alongside accuracy, precision, recall and F1-score. ROC curves and AUC were computed from predicted probabilities, and trained models and the scaler were saved with joblib.

Pipeline 2: Advanced Models

The second pipeline adds five models. Random Forest uses 200 trees. Histogram-based Gradient Boosting (scikit-learn HistGradientBoostingClassifier) uses 150 iterations, a learning rate of 0.1 and up to 31 leaf nodes. XGBoost uses 200 trees with a maximum depth of 6, a learning rate of 0.1 and a subsample ratio of 0.8. LinearSVC uses C = 1.0 with up to 5,000 iterations, and the MLP uses two hidden layers of 64 and 32 units with ReLU activation, the Adam solver and early stopping. Because the MLP receives three independent pixel features rather than whole images, it is a tabular neural classifier and not a convolutional segmentation network.

Model	Key settings (initial comparison)	Scaled input
Logistic Regression	max_iter = 1000, random_state = 42	Yes
Decision Tree	default parameters, random_state = 42	No
KNN	n_neighbors = 5, algorithm = kd_tree	Yes
Random Forest	n_estimators = 200, random_state = 42	No

Model	Key settings (initial comparison)	Scaled input
Gradient Boosting	max_iter = 150, learning_rate = 0.1, max_leaf_nodes = 31	No
XGBoost	n_estimators = 200, max_depth = 6, learning_rate = 0.1, subsample = 0.8	No
LinearSVC	C = 1.0, max_iter = 5000	Yes
MLP	hidden layers (64, 32), ReLU, Adam, early stopping	Yes

Table 4: Model Configurations Used in the Initial Comparison

Training, Cross-Validation and Evaluation

Models were evaluated with accuracy, precision, recall, F1-score, confusion matrices and ROC-AUC, treating skin as the positive class. Hyperparameter optimisation used RandomizedSearchCV with five-fold StratifiedGroupKFold cross-validation, F1-score as the optimisation criterion and five candidate configurations per model, giving 25 fits per model. The training partition contains 41,148 unique BGR groups, and each group stays within a single fold. Class-imbalance experiments were implemented with imbalanced-learn pipelines so that resampling is applied only to the training portion of each fold.

System Architecture

The overall workflow is organised as ten connected stages. Data loading and exploratory analysis feed a group-aware split and scaling stage. The prepared data is then used by the baseline and advanced modelling stages, whose results are refined by cross-validation, tuning and imbalance experiments. Feature-importance analysis explains the strongest models, and a final cross-validated comparison selects the model that is retrained and evaluated on the held-out test set. Figure 2 summarises this flow.

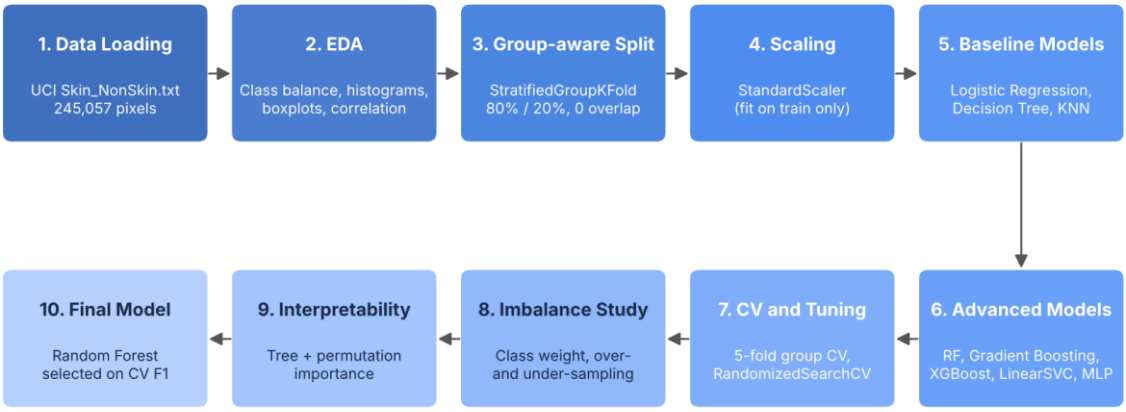


Figure 2: Experimental Workflow of the Skin Segmentation Study

Feature Importance and Interpretability Module

Because the models are intended to be understood and not only scored, an interpretability stage was added. Random Forest and XGBoost were chosen because they performed strongly and provide built-in tree importance scores. Permutation importance was computed on the test set with F1-score as the metric and ten repeats, measuring how much the F1-score falls when a single feature is shuffled.

Built-in importance reflects how often and how effectively a feature is used in the tree splits, whereas permutation importance measures the loss in predictive performance when the feature is disrupted. Using both gives a more reliable picture. Both are model-specific predictive diagnostics and should not be read as causal explanations. Because the colour channels are correlated, particularly blue and green, the importance of an individual channel may be shared with, or masked by, its correlated neighbours, and shuffling a single channel can create colour combinations that rarely occur naturally.

Results and Discussion

Exploratory Data Analysis

Figure 3 shows that the dataset is imbalanced, with skin pixels forming the minority class. Figure 4 compares the distribution of each colour channel by class. The blue and green distributions overlap substantially, so neither channel alone separates skin from non-skin. The red channel shows a clearer difference, with skin pixels concentrated at higher red intensities, although the distributions still overlap and a single red threshold would not be sufficient.

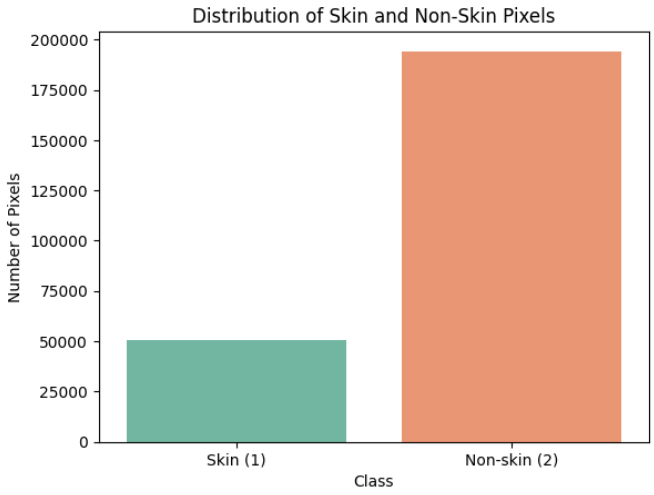


Figure 3: Distribution of Skin and Non-Skin Pixels

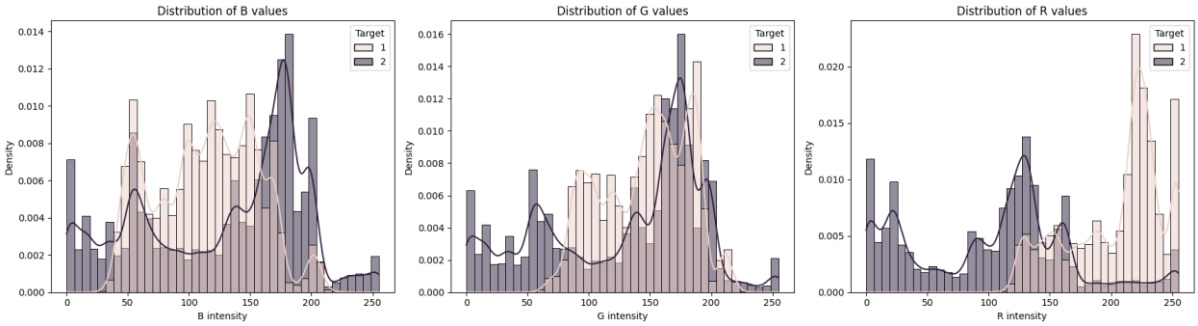


Figure 4: Distribution of Individual Colour Channels by Class

The boxplots in Figure 5 support this observation. The blue-channel median is higher for non-skin pixels, the green medians are comparatively similar across classes, and the red channel shows a marked separation, with a much higher median for skin. The correlation analysis in Figure 6 shows that blue and green are strongly correlated (0.86), green and red moderately to strongly correlated (0.66), and blue and red moderately correlated (0.50). All three channels were retained for modelling.

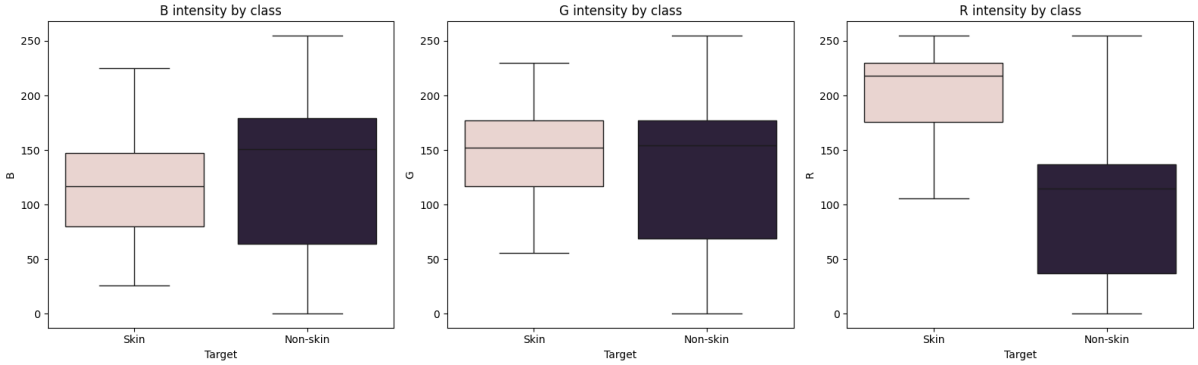


Figure 5: Boxplots of B, G and R Intensity by Class

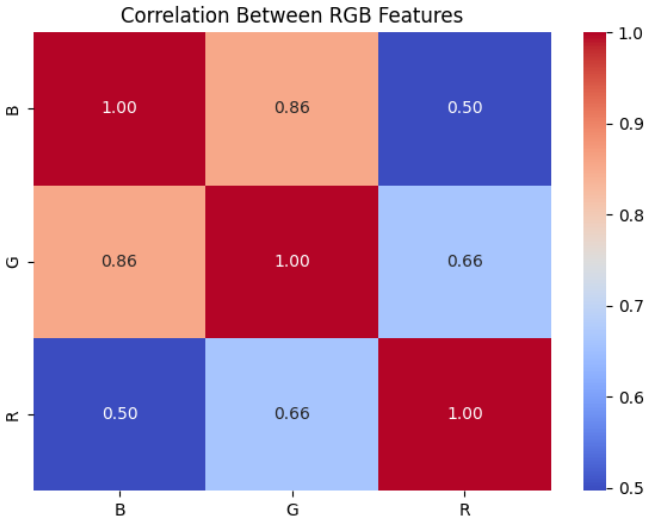


Figure 6: Correlation Between RGB Features

Baseline Model Results

Table 5 reports the baseline results on the held-out test set. Logistic Regression reached 91.85% accuracy and an F1-score of 0.8140, with 2,341 false positives and 1,629 false negatives,

confirming that a linear boundary cannot represent the skin region well. The Decision Tree reached 99.91% accuracy and an F1-score of 0.9978, with only 33 false positives and 12 false negatives. KNN gave the strongest baseline: 99.93% accuracy, 99.69% precision, 99.98% recall and an F1-score of 0.9984, with 32 false positives and just 2 false negatives. Its prediction time of 0.643 seconds, however, was far longer than that of the other two models.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC	Train (s)	Predict (s)
Logistic Regression	0.9185	0.7877	0.8421	0.8140	0.9499	0.300	0.001
Decision Tree	0.9991	0.9968	0.9988	0.9978	0.9991	0.479	0.003
KNN	0.9993	0.9969	0.9998	0.9984	0.9998	0.318	0.643

Table 5: Baseline Model Performance on the Test Set

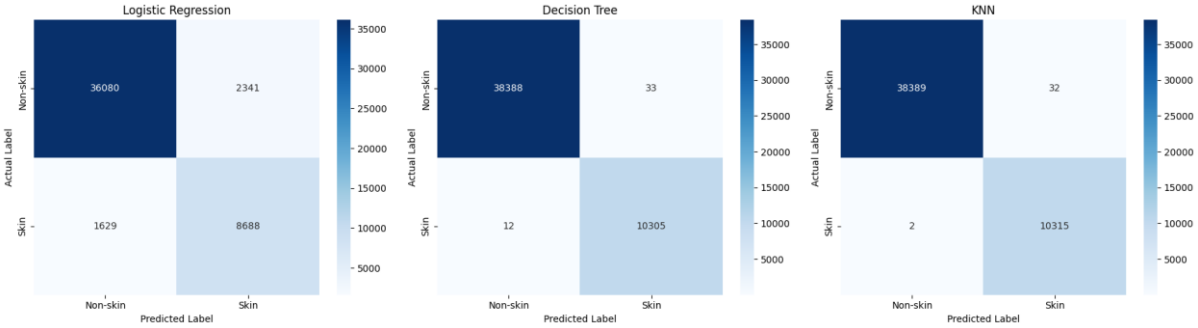


Figure 7: Confusion Matrices of the Baseline Models

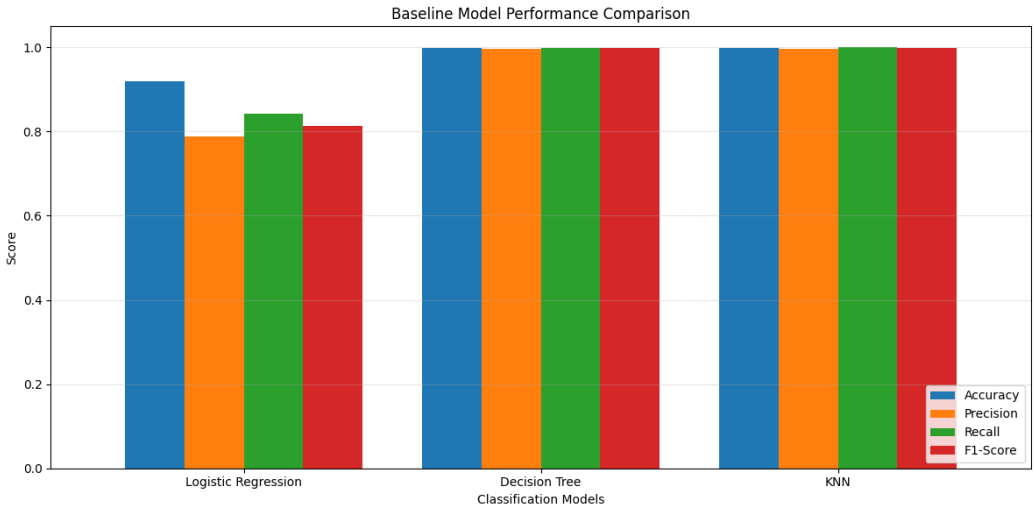


Figure 8: Baseline Performance Comparison

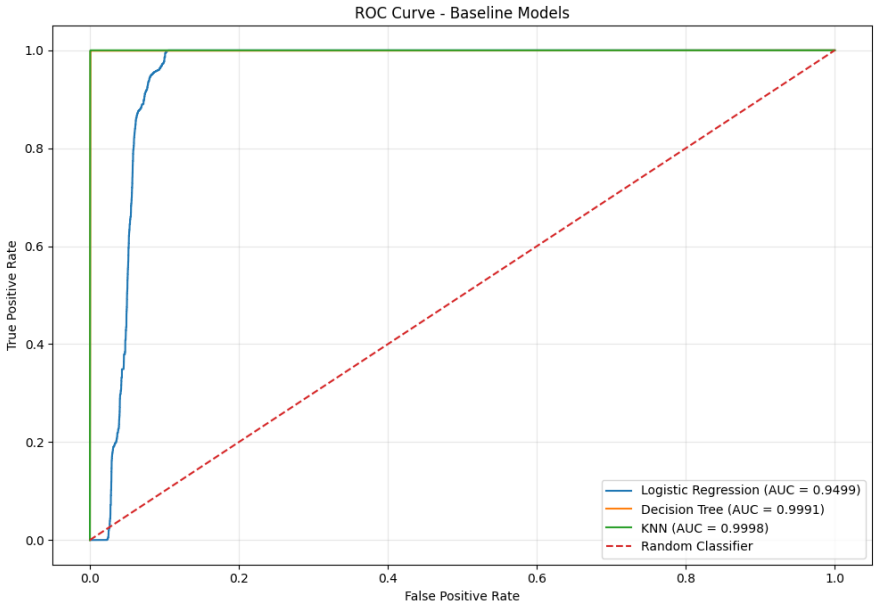


Figure 9: ROC Curves of the Baseline Models

Advanced Model Results

Table 6 reports the initial comparison of the five advanced models. Random Forest, Gradient Boosting and XGBoost all reached 99.92% accuracy and F1-scores of about 0.998, and the MLP was close behind at 99.91% accuracy and an F1-score of 0.9978. Gradient Boosting had the highest recall (0.9997), missing only 3 skin pixels, while Random Forest had the highest precision (0.9970). LinearSVC reached only 92.46% accuracy and an F1-score of 0.8309, similar to Logistic Regression, again indicating that a linear separating boundary is unsuitable. In the combined comparison of all eight models, KNN remained the strongest on the initial test split (F1 = 0.998355), which motivated the cross-validated comparison reported later instead of choosing a model from a single test score.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC	Train (s)	Predict (s)
Random Forest	0.99924	0.99700	0.99942	0.99821	0.999981	10.965	0.1637
Gradient Boosting	0.99922	0.99662	0.99971	0.99816	0.999980	1.772	0.4669
XGBoost	0.99922	0.99691	0.99942	0.99816	0.999973	1.391	0.1037
MLP	0.99908	0.99633	0.99932	0.99782	0.999965	18.869	0.0498
LinearSVC	0.92462	0.79118	0.87477	0.83088	0.946430	0.072	0.0007

Table 6: Advanced Model Performance on the Test Set

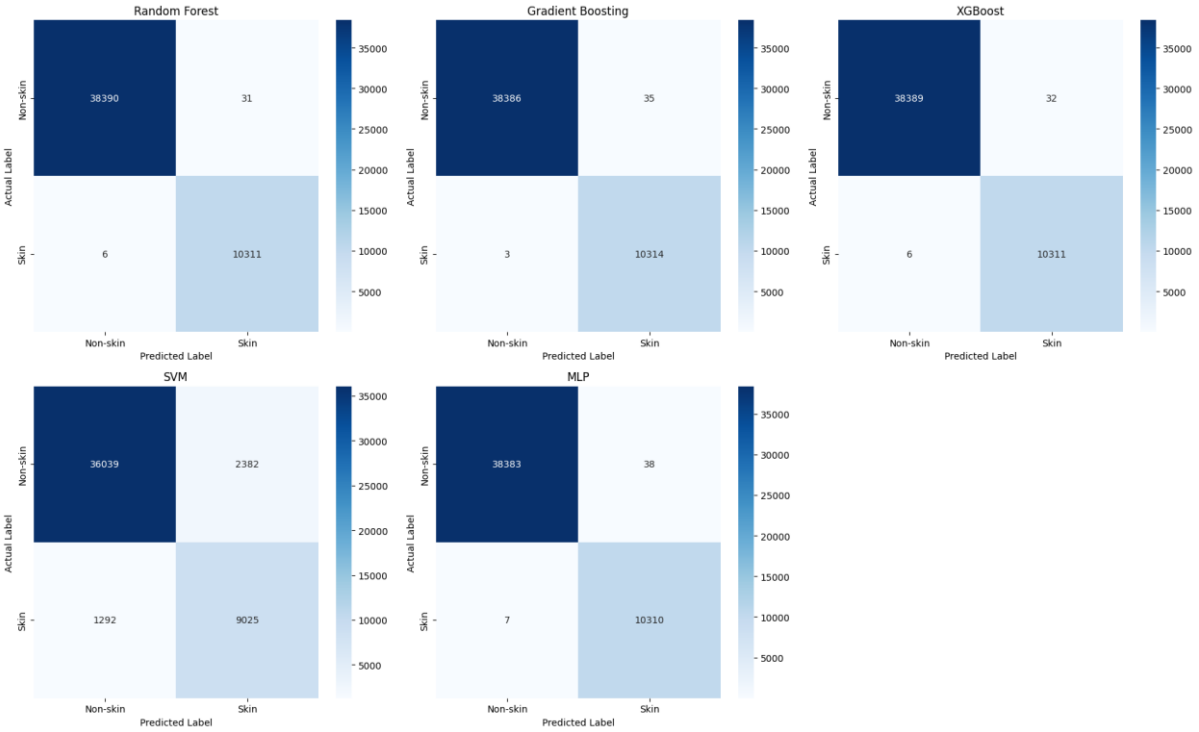


Figure 10: Confusion Matrices of the Advanced Models (SVM = LinearSVC)

Table 7 gives the confusion-matrix counts for all eight models on the 48,738 test pixels. The best models make only a few dozen errors in total, whereas the two linear models make thousands.

Model	True Negative	False Positive	False Negative	True Positive
Logistic Regression	36,080	2,341	1,629	8,688
Decision Tree	38,388	33	12	10,305
KNN	38,389	32	2	10,315
Random Forest	38,390	31	6	10,311
Gradient Boosting	38,386	35	3	10,314
XGBoost	38,389	32	6	10,311
MLP	38,383	38	7	10,310
LinearSVC	36,039	2,382	1,292	9,025

Table 7: Confusion-Matrix Counts on the Test Set (Skin = Positive Class)

Cross-Validation and Hyperparameter Tuning

Table 8 lists the best cross-validated F1-score and the selected parameters for each tuned model. Random Forest and XGBoost reached the highest tuned CV F1-score (0.99875). The LinearSVC search emitted a convergence warning from the liblinear solver, which should be kept in mind when interpreting its tuned result.

Model	Best CV F1	Tuning time (s)	Selected parameters
Random Forest	0.99875	228.57	n_estimators = 100, min_samples_split = 10, min_samples_leaf = 2, max_features = sqrt, max_depth = 30

Model	Best CV F1	Tuning time (s)	Selected parameters
Gradient Boosting	0.99862	64.03	max_leaf_nodes = 31, max_iter = 150, learning_rate = 0.1, l2_regularization = 0
XGBoost	0.99875	64.17	subsample = 0.8, n_estimators = 200, max_depth = 7, learning_rate = 0.1, colsample_bytree = 1.0
LinearSVC	0.85618	45.12	loss = hinge, class_weight = balanced, C = 10
MLP	0.99857	302.64	hidden_layer_sizes = (64, 32), activation = tanh, alpha = 0.001, learning_rate_init = 0.001

Table 8: Hyperparameter Search Results (5-fold StratifiedGroupKFold)

Table 9 compares default and tuned models on the test set. Tuning did not guarantee better results. Random Forest and Gradient Boosting kept essentially the same F1-score, XGBoost and MLP showed very small decreases (about 0.0001), and only LinearSVC improved, by 0.0237, because the balanced class weighting raised its recall to 1.0000 at the cost of precision (0.7462). The default configurations of the strongest tree-based models were therefore already highly competitive on this dataset.

Model	Default F1	Tuned F1	Default AUC	Tuned AUC	Tuned Precision	Tuned Recall
Random Forest	0.998209	0.998209	0.999981	0.999990	0.99710	0.99932
Gradient Boosting	0.998161	0.998161	0.999980	0.999980	0.99662	0.99971
XGBoost	0.998161	0.998064	0.999973	0.999981	0.99671	0.99942
LinearSVC	0.830878	0.854622	0.946430	0.935744	0.74615	1.00000
MLP	0.997822	0.997727	0.999965	0.999922	0.99575	0.99971

Table 9: Default versus Tuned Test Performance

Class Imbalance Experiments

With a non-skin to skin ratio of 3.84 to 1 in the training data, class weighting, random oversampling and random undersampling were compared against the original distribution for Logistic Regression, Random Forest and XGBoost (Table 10). For Logistic Regression, class weighting and oversampling raised the test F1-score from 0.8140 to 0.8510 and the recall from 0.8421 to 0.9673, but lowered precision from 0.7877 to 0.7596, so more skin pixels were found at the cost of more false alarms. For Random Forest, the effect was marginal, with oversampling giving the highest test F1-score (0.9984) and undersampling reducing it (0.9970). XGBoost performed best with the original distribution (0.9981), and undersampling reduced it most. Balancing should therefore be justified by validation results and by the relative cost of the two error types and not applied automatically.

Model	Strategy	CV F1	Precision	Recall	Test F1	ROC-AUC
Logistic Regression	Original	0.81240	0.78774	0.84211	0.81402	0.94989
Logistic Regression	Class weight	0.84469	0.75963	0.96734	0.85099	0.94299
Logistic Regression	Oversampling	0.84431	0.75963	0.96734	0.85099	0.94300
Logistic Regression	Undersampling	0.84543	0.75864	0.96578	0.84977	0.94341
Random Forest	Original	0.99886	0.99700	0.99942	0.99821	0.99998

Model	Strategy	CV F1	Precision	Recall	Test F1	ROC-AUC
Random Forest	Class weight	0.99878	0.99720	0.99952	0.99835	0.99998
Random Forest	Oversampling	0.99875	0.99710	0.99971	0.99840	0.99997
Random Forest	Undersampling	0.99715	0.99441	0.99961	0.99700	0.99997
XGBoost	Original	0.99853	0.99681	0.99942	0.99811	0.99997
XGBoost	Class weight	0.99851	0.99623	0.99952	0.99787	0.99997
XGBoost	Oversampling	0.99848	0.99642	0.99952	0.99797	0.99997
XGBoost	Undersampling	0.99735	0.99393	0.99961	0.99676	0.99996

Table 10: Effect of Imbalance-Handling Strategies (CV on training folds; metrics on the test set)

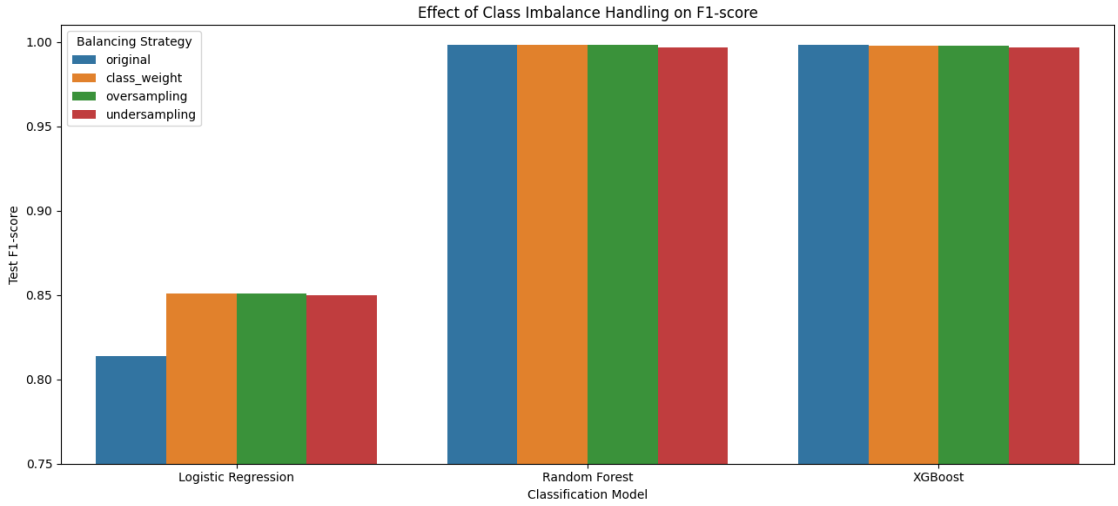


Figure 11: Effect of Imbalance Handling on Test F1-score

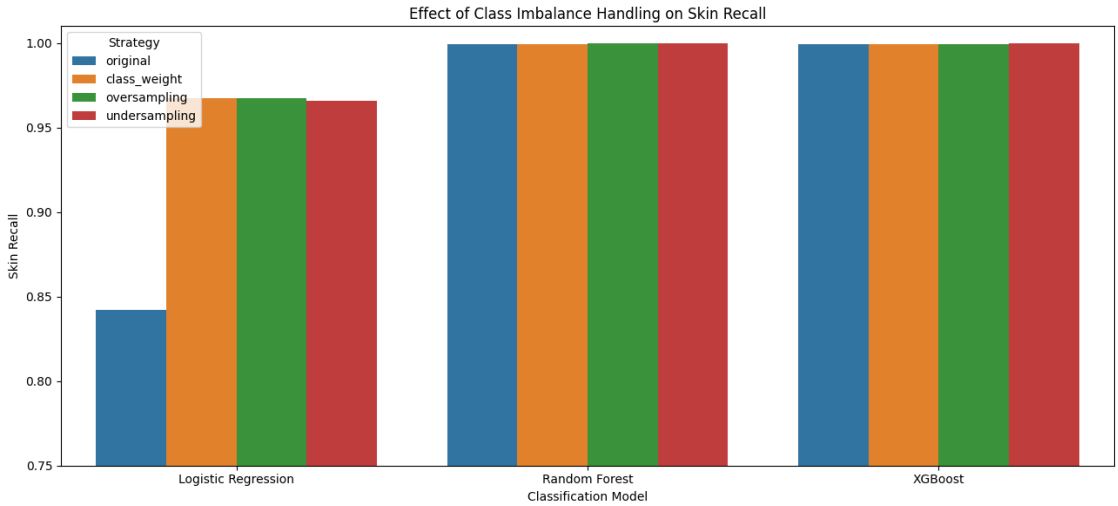


Figure 12: Effect of Imbalance Handling on Skin Recall

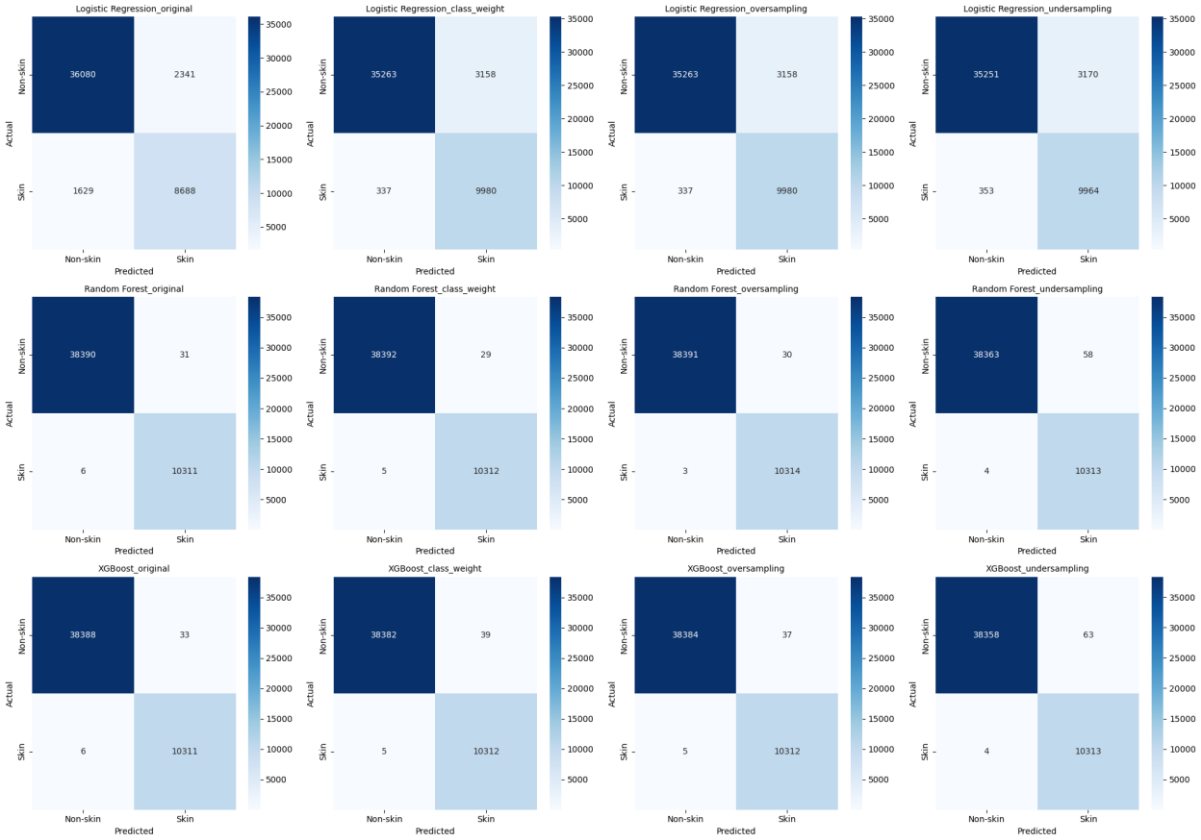


Figure 13: Confusion Matrices for All Imbalance Experiments

Feature Importance Results

Table 11 and Figures 14 to 17 show that red is the dominant feature under both importance measures and both models. Random Forest assigned about 54.7% of its tree importance to red, 23.4% to green and 21.9% to blue, while XGBoost assigned 69.4%, 15.0% and 15.7% respectively. Under permutation importance, shuffling red caused the largest mean drop in F1-score (0.686 for Random Forest and 0.660 for XGBoost), followed by green and blue, with small standard deviations (between 0.0036 and 0.0051). Green and blue remain useful, since shuffling either of them also reduces the F1-score substantially, so no channel should be discarded.

Feature	RF tree	XGB tree	RF permutation (F1 drop)	XGB permutation (F1 drop)
Red (R)	0.546983	0.693543	0.686273	0.659965
Green (G)	0.234253	0.149719	0.516949	0.486974
Blue (B)	0.218765	0.156738	0.480099	0.445957

Table 11: Feature Importance Comparison

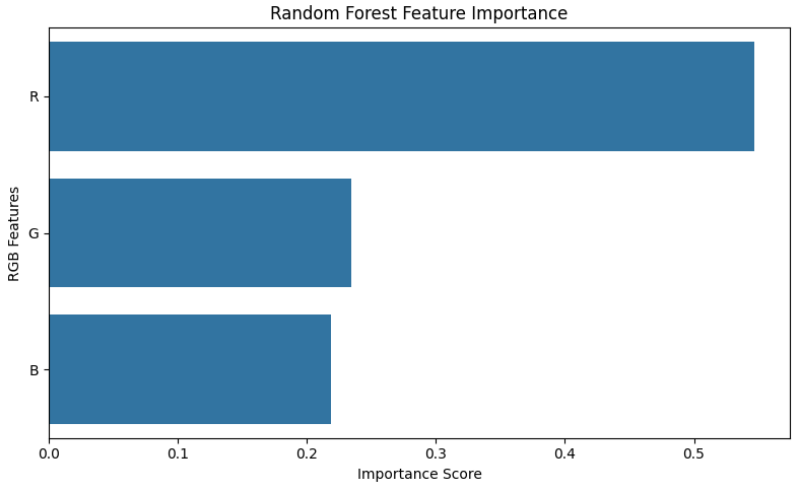


Figure 14: Random Forest Feature Importance

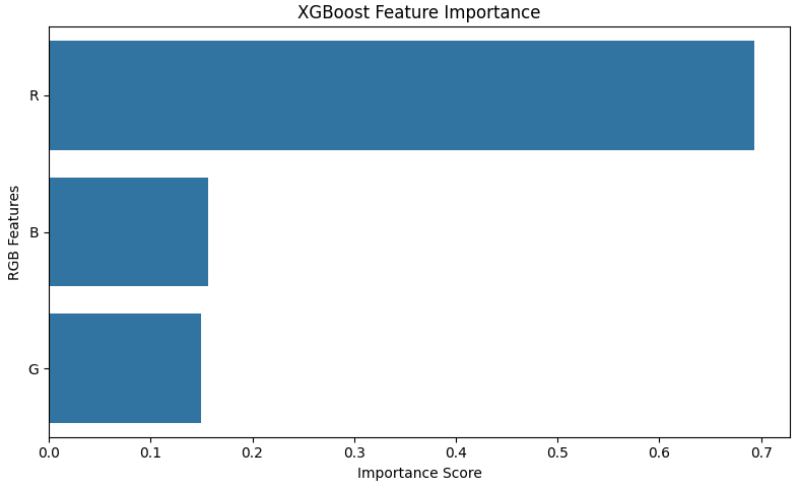


Figure 15: XGBoost Feature Importance

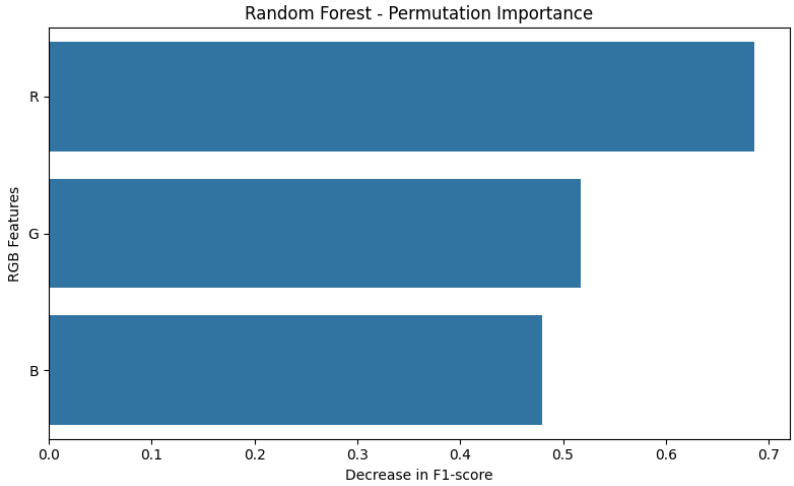


Figure 16: Random Forest Permutation Importance

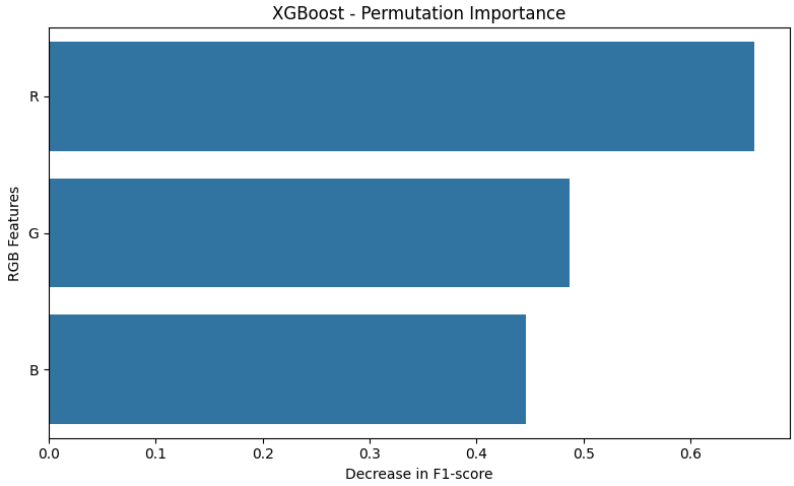


Figure 17: XGBoost Permutation Importance

Final Model Comparison and Selection

To select the final model, all eight classifiers were compared under the same five-fold StratifiedGroupKFold cross-validation on the training partition (Table 12). Random Forest obtained the highest mean F1-score (0.998865), the highest accuracy (0.999531), the highest precision (0.998045) and the highest ROC-AUC (0.999978), with a low F1 standard deviation of 0.000198. KNN was extremely close (F1 = 0.998851) and had slightly higher recall and a shorter cross-validation time, but the F1 difference between the two is only 0.000014, so they are practically indistinguishable on this dataset. LinearSVC and Logistic Regression were far behind, with F1-scores of about 0.83 and 0.81 and much larger fold-to-fold variation.

Model	CV Acc.	CV Prec.	CV Recall	CV F1 mean	CV F1 std	CV AUC	Time (s)
Random Forest	0.999531	0.998045	0.999686	0.998865	0.000198	0.999978	61.99
KNN	0.999524	0.997826	0.999879	0.998851	0.000185	0.999821	14.24
XGBoost	0.999503	0.997978	0.999635	0.998805	0.000267	0.999970	18.78
Gradient Boosting	0.999427	0.997804	0.999443	0.998623	0.000231	0.999955	19.00
MLP	0.999275	0.997110	0.999399	0.998253	0.000514	0.999952	62.14
Decision Tree	0.998764	0.996397	0.997623	0.997008	0.000905	0.998473	11.88
LinearSVC	0.925819	0.798934	0.856176	0.826511	0.009424	0.951016	11.54
Logistic Regression	0.921026	0.797179	0.828363	0.812398	0.009205	0.953803	29.49

Table 12: Five-Fold Group-Aware Cross-Validation Comparison of All Models

Random Forest was therefore selected as the final model because it achieved the best cross-validated F1-score, precision and ROC-AUC while retaining very high recall and stable performance across folds, and because it supports the feature-importance analysis described above. This is not a claim that it is meaningfully superior to KNN in every respect: KNN is faster to cross-validate and marginally higher in recall, but its prediction is slower on large inputs because it must search the stored training data for every query.

Final Model Evaluation

The selected Random Forest (200 trees, random seed 42) was retrained on the full training partition and evaluated on the held-out test set. It achieved 99.92% accuracy, 99.70% precision, 99.94% recall, an F1-score of 0.9982 and a ROC-AUC of 0.99998. Figure 18 shows that only 31 of the 38,421 non-skin pixels were misclassified as skin and only 6 of the 10,317 skin pixels were missed. The model was saved as final_skin_segmentation_model.pkl.

Metric	Value
Accuracy	0.999241
Precision (skin)	0.997003
Recall (skin)	0.999418
F1-score (skin)	0.998209
ROC-AUC	0.999981
False positives / False negatives	31 / 6

Table 13: Final Random Forest Performance on the Held-Out Test Set

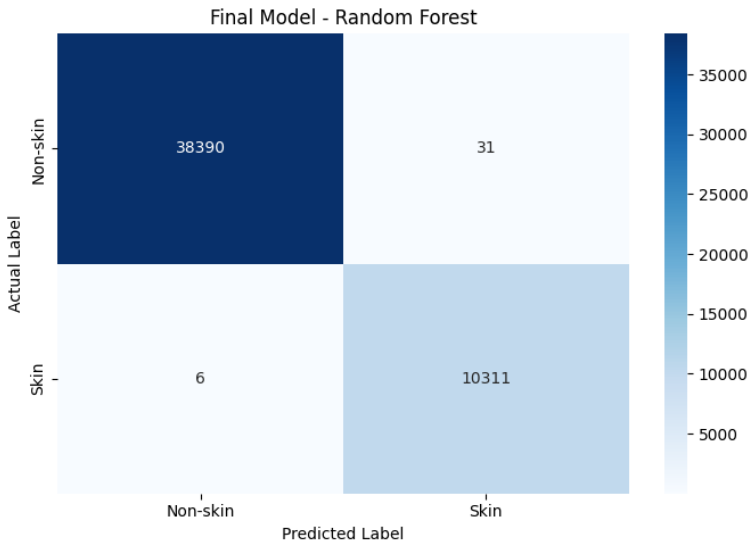


Figure 18: Confusion Matrix of the Final Random Forest Model

Limitations and Threats to Validity

The very high scores should be interpreted carefully. First, the classifier uses only three colour values per pixel. It has no information about texture, neighbouring pixels, position or image context, so pixels with similar colours in different objects cannot be distinguished, and a predicted mask can be scattered and lack coherent boundaries. Second, the dataset provides no skin-tone or demographic annotations, so these experiments cannot establish fairness or equal error rates across skin tones. Third, RGB values change with illumination, shadows, camera sensors and white balance, and no systematic lighting study was run.

Fourth, all results come from one dataset and one group-aware partition. Grouping identical BGR triplets removes one form of leakage but is not a substitute for testing on an independently collected dataset. Fifth, the 11 triplets with conflicting labels show that colour alone does not determine the class perfectly. Finally, the held-out test set was used in several comparisons

(baseline, advanced, tuned and imbalance experiments), although the final model was chosen using cross-validation on the training partition only. A stricter protocol would lock the configuration using training-only validation and evaluate once on a separate test set.

Conclusion

This project investigated pixel-level skin and non-skin classification using the UCI Skin Segmentation dataset. The work covered exploratory analysis, preprocessing, a group-aware split that was verified to have no overlapping colour groups, baseline and advanced classification, ROC-AUC evaluation, cross-validation, hyperparameter tuning, class-imbalance experiments and feature-importance analysis.

Eight algorithms were compared. The baseline experiments showed that Decision Tree and KNN substantially outperformed Logistic Regression, and the advanced experiments showed that Random Forest, Gradient Boosting, XGBoost and MLP also achieved very high performance, while LinearSVC remained limited by its linear boundary. In the final five-fold group-aware comparison, Random Forest obtained the highest mean F1-score (0.998865), narrowly ahead of KNN (0.998851), and was selected as the final model. On the held-out test set it reached 99.92% accuracy, an F1-score of 0.9982 and a ROC-AUC of 0.99998, with only 37 misclassified pixels out of 48,738.

Feature-importance analysis consistently identified red intensity as the most influential channel, while green and blue still contributed useful information. Class weighting and oversampling substantially improved the recall of Logistic Regression but changed the ensemble models only marginally. Overall, classical machine learning classifies the RGB observations in this dataset with very high measured accuracy. However, the study does not establish robustness to changing lighting, fairness across skin tones, or generalisation to independent image datasets; these remain the natural next steps.

Future Scope

The most important extension is evaluation on independent, annotated image data. A separate skin-segmentation dataset that was never used for training or model selection would test generalisation to new cameras and environments. If the data consists of full images, the split should be done at image level so that pixels from the same image never appear in both training and evaluation data.

A second direction is fairness analysis. This requires a dataset with skin-tone annotations or a defensible grouping protocol, so that recall, false-positive rate and false-negative rate can be reported for each group together with sample sizes. The current UCI table does not contain such annotations, which is why this report draws no fairness conclusion.

Robustness to illumination can be studied by applying controlled brightness, shadow and colour-temperature changes and measuring performance under each. Alternative colour spaces, such as HSV and YCbCr, can be compared with raw RGB using the same split, folds, metrics and tuning budget. Spatial context could be added through neighbourhood features or post-processing, or through a convolutional segmentation network such as a U-Net trained on full

images with pixel-level masks. Finally, a real-time prototype could accept an image, extract pixel features, apply the trained Random Forest and reconstruct a skin mask, with testing of latency, memory use and image resolution.

References

1. UCI Machine Learning Repository. Skin Segmentation Dataset (Skin_NonSkin.txt), contributed by R. Bhatt and A. Dhall. https://archive.ics.uci.edu/ml/machine-learning-databases/00229/Skin_NonSkin.txt
2. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
3. Chen, T., and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.
4. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
5. Cover, T., and Hart, P. (1967). Nearest Neighbor Pattern Classification. IEEE Transactions on Information Theory, 13(1), 21-27.
6. Jones, M. J., and Rehg, J. M. (2002). Statistical Color Models with Application to Skin Detection. International Journal of Computer Vision, 46(1), 81-96.
7. Kakumanu, P., Makrogiannis, S., and Bourbakis, N. (2007). A Survey of Skin-Color Modeling and Detection Methods. Pattern Recognition, 40(3), 1106-1122.
8. Vezhnevets, V., Sazonov, V., and Andreeva, A. (2003). A Survey on Pixel-Based Skin Color Detection Techniques. Proceedings of GraphiCon.
9. Buolamwini, J., and Gebru, T. (2018). Gender Shades: Intersectional Accuracy Disparities in Commercial Gender Classification. Proceedings of Machine Learning Research, 81, 1-15.

GitHub Link: https://github.com/lakshmi-srujana/MLproject.git